



Topics in Software Engineering – Software Reengineering

By

Dr. Junaid Akram

Assistant Professor, Department of Computer Science COMSATS (Lahore)

Software Change & Maintenance



Software Change

Software change is inevitable

- New **requirements emerge** when the software is used
- The business environment changes
- **Errors** must be repaired
- **New equipment** must be accommodated
- The **performance** or reliability may have to be improved

A key problem for organisations is implementing and managing change to their legacy systems

Software Change Strategies

Software maintenance

- Changes are made in response to changed requirements, but the fundamental software structure remains stable

Architectural transformation

- The architecture of the system is modified generally from a centralised architecture to a distributed architecture

Software re-engineering

- No new functionality is added to the system but it is restructured and reorganised to facilitate future changes

These strategies may be applied separately or together

Change Requests



Change requests are requests for system changes from **users**, **customers** or **management**



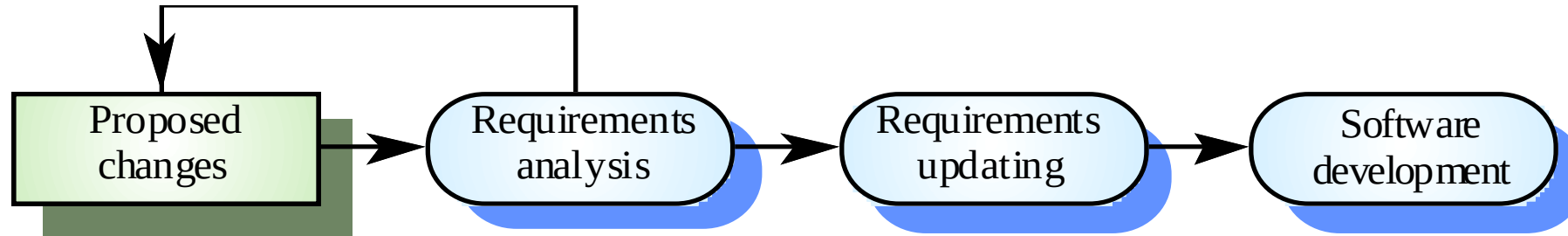
In principle, all change requests should be carefully analysed as part of the maintenance process and then implemented



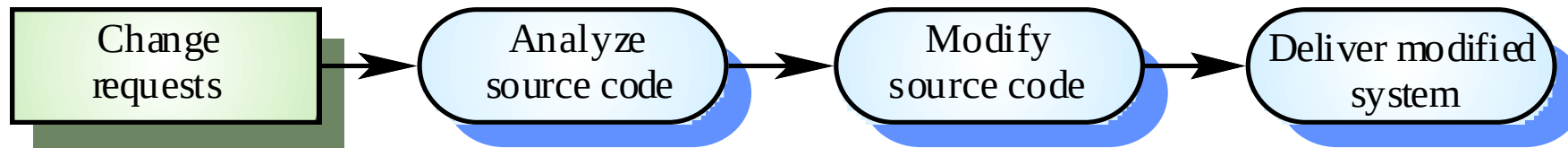
In practice, some change requests must be implemented urgently

- Fault repair
- Changes to the system's environment
- Urgently required business changes

Change Implementation

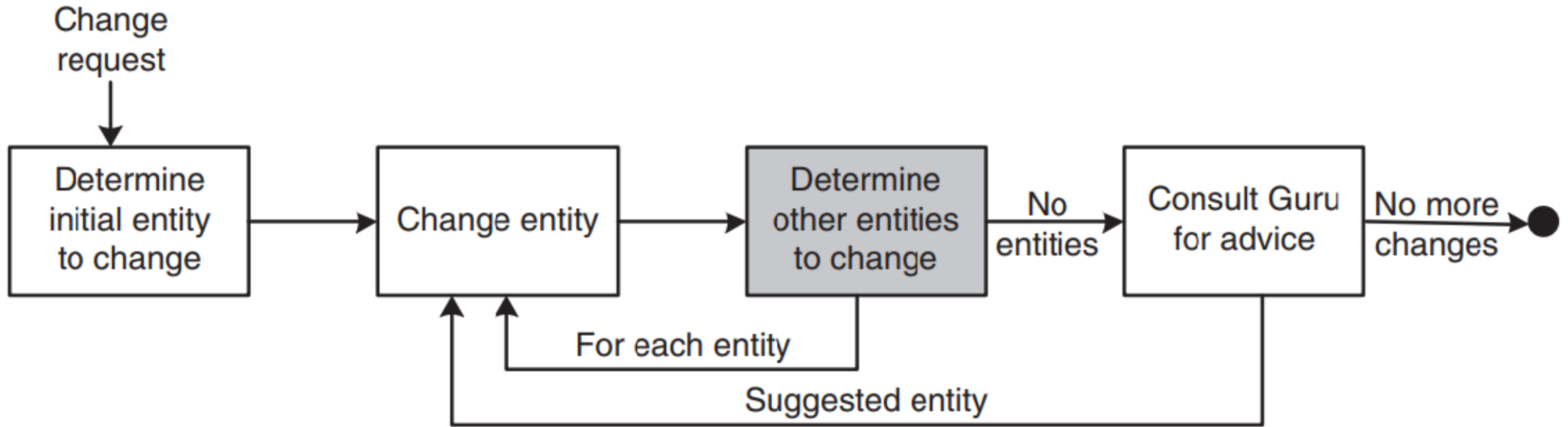


Emergency Change Implementation



Change Propagation Model

Ref: © 2004 IEEE



Change Request Field Summary

Field Name	Description
change_request_id	A unique identifier of the CR
title	A concise summary of the CR
description	A short description of the CR
maintenance_type	Classification of the maintenance type in terms of a member of {Corrective, Adaptive, Perfective, Preventive}
product	Product name
component	Component where the change is needed, or where the problem occurred
state	Present state of the CR in terms of a member of {Submit, Review, Analysis, Commit, Implementation, Verification, Closed, Declined}
customer	Name of the customer making the CR
problem_origin	The origin of the problem
impacts	Components that are affected by a change and its ripple effect
resolution	Documentation of what was changed, how, and why
note	Additional information provided by the submitter for subsequent decision making
software_release	The version number of the product release in which the CR is likely to be effective
committed_release	The version number of the product release in which the CR will be effective
priority	Priority of CR, which is an element of a set, namely, {normal, high}
severity	Severity of CR, which is an element of a set, namely, {normal, critical}

Software Maintenance



Concepts and Methods

Software Maintenance

- Modifying a program after it has been put into use
- The term is mostly used for **changing custom software**. Generic software products are said to **evolve** to create new versions.
- Maintenance **does not normally involve major changes** to the system's architecture
- Changes are implemented by modifying existing components or adding new components to the system
- The ISO/IEC 14764 standard defines software maintenance as "... the activities required to provide cost-effective support to a software system.
- Activities are performed during the pre-delivery stage as well as the post-delivery stage

Why Modifications are Required



Market Conditions - Policies, which changes over the time, such as taxation and newly introduced constraints like, how to maintain bookkeeping, may trigger need for modification.



Client Requirements - Over the time, customer may ask for new features or functions in the software.



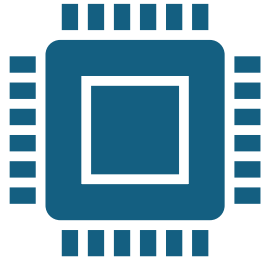
Host Modifications - If any of the hardware and/or platform (such as operating system) of the target host changes, software changes are needed to keep adaptability.



Organization Changes - If there is any business level change at client end, such as reduction of organization strength, acquiring another company, organization venturing into new business, need to modify in the original software may arise.

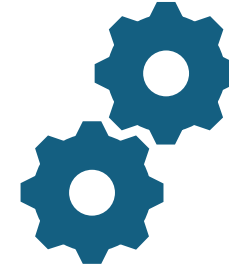
A major difference exists between software maintenance and software development: **maintenance is event driven, whereas development is requirements driven**

Maintenance Category



Maintenance to repair software faults

- Changing a system to correct deficiencies in the way meets its requirements
- Coding errors are usually relatively cheap to correct; design errors are more expensive as they may involve rewriting several program components.
- Requirements errors are the most expensive to repair because of the extensive system redesign which may be necessary.



Maintenance to adapt software to a different operating environment

- This type of maintenance is required when some aspect of the system's environment such as the hardware, the platform operating system, or other support software changes.
- The application system must be modified to adapt it to cope with these environmental changes.

Types of Maintenance

- **Corrective Maintenance** -

This includes modifications and updates done in order to correct or fix problems, which are either discovered by user or concluded by user error reports.

- **Adaptive Maintenance** -

This includes modifications and updates applied to keep the software product up-to date and tuned to the ever changing world of technology and business environment.

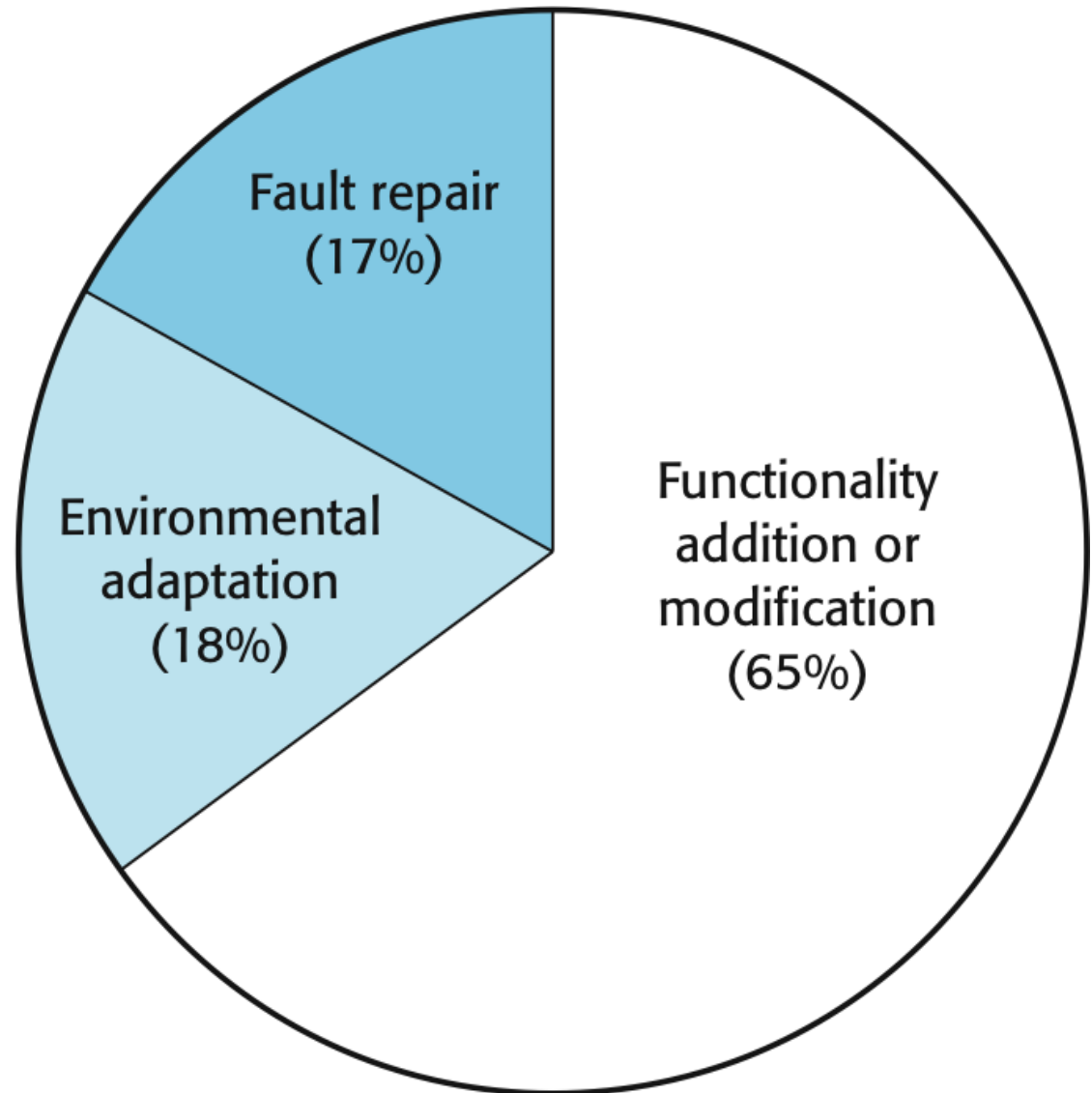
- **Perfective Maintenance** -

This includes modifications and updates done in order to keep the software usable over long period of time. It includes new features, new user requirements for refining the software and improve its reliability and performance.

- **Preventive Maintenance** -

This includes modifications and updates to prevent future problems of the software. It aims to attend problems, which are not significant at this moment but may cause serious issues in future.

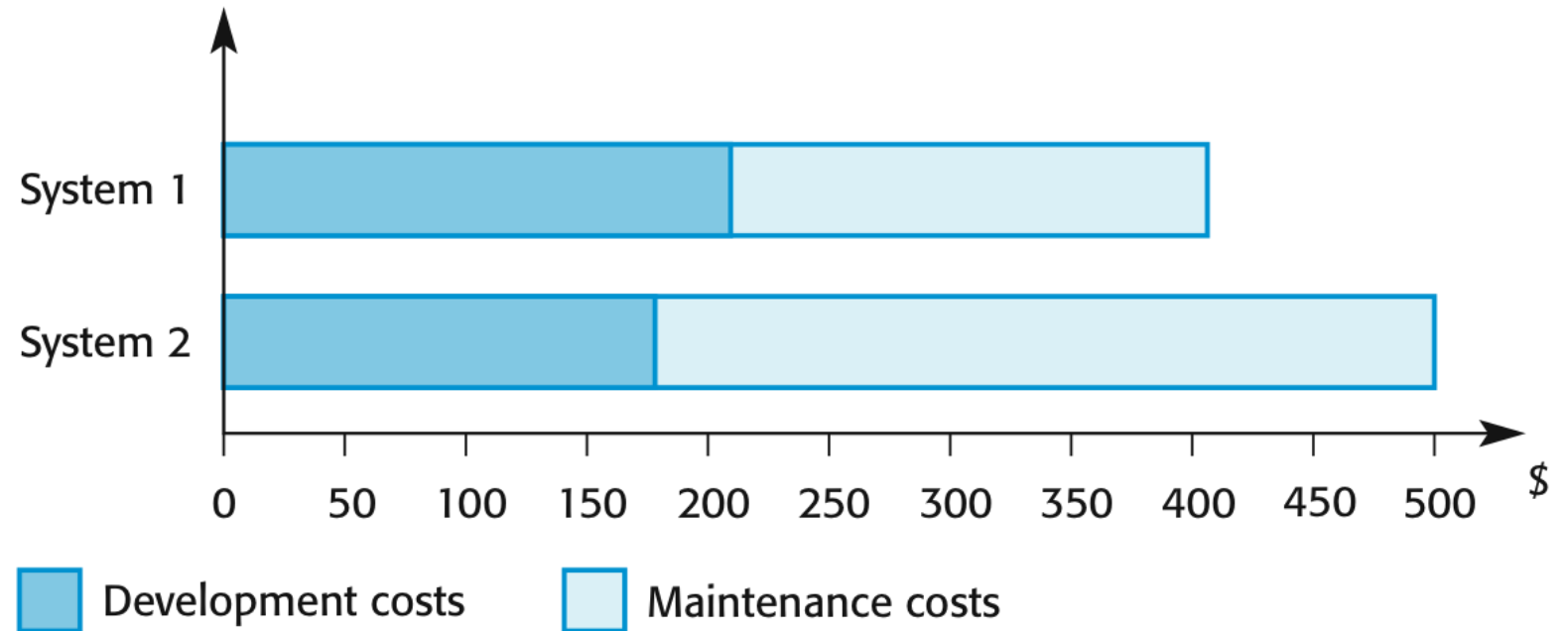
Maintenance Effort Distribution



Maintenance Costs

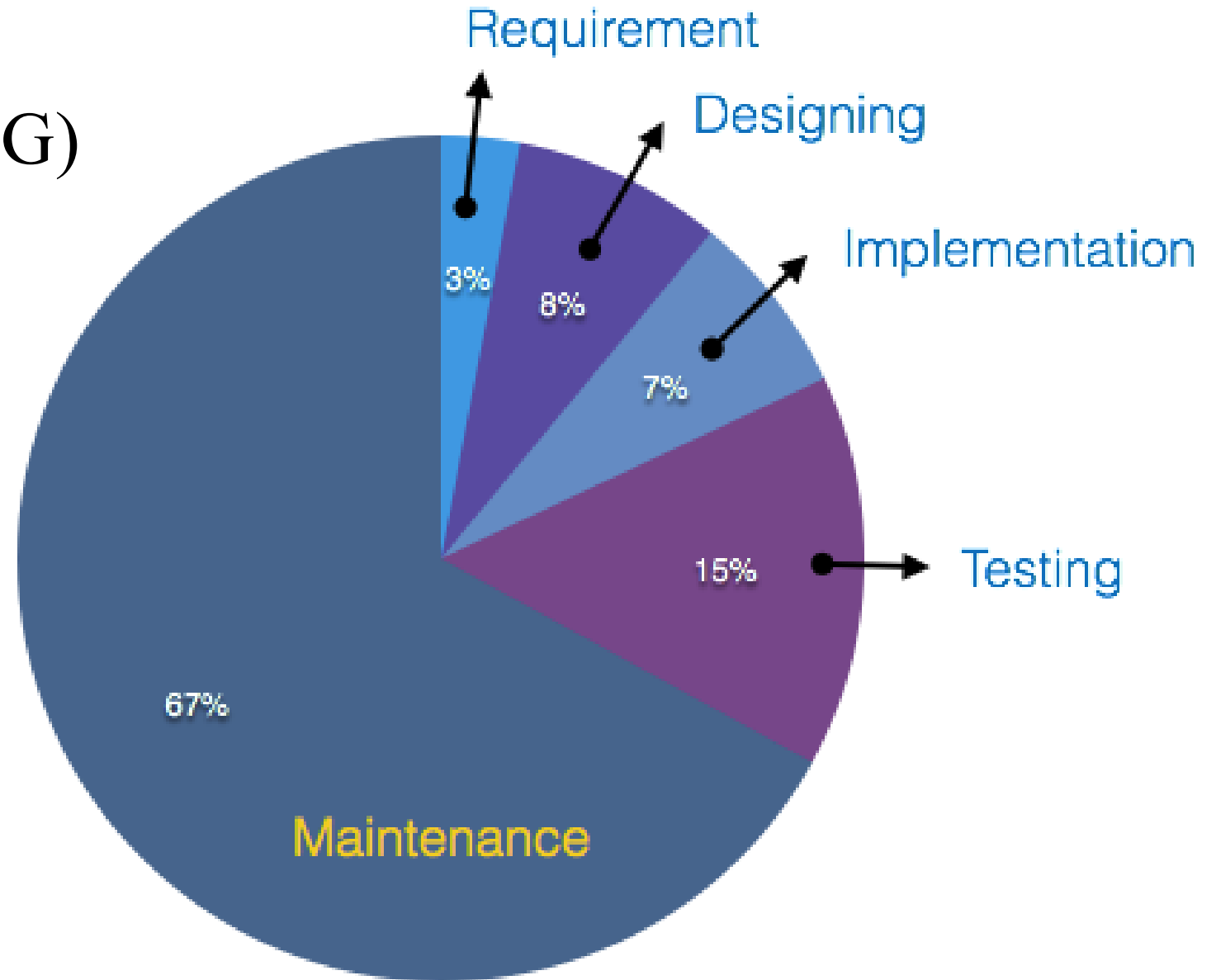
- Usually greater than development costs
- Increases as software is maintained. Maintenance corrupts the software structure so makes further maintenance more difficult.
- Ageing software can have high support costs (e.g., old languages, compilers etc.)

Development and maintenance costs



Cost of Maintenance (AVG)

- Reports suggest that the cost of maintenance is high. A study on estimating software maintenance found that the cost of maintenance is as high as 67% of the cost of entire software process cycle.



Maintenance Cost Factors

Team stability

- Maintenance costs are reduced if the same staff are involved with them for some time

Contractual responsibility

- The developers of a system may have no contractual responsibility for maintenance so there is no incentive to design for future change

Staff skills

- Maintenance staff are often inexperienced and have limited domain knowledge

Program age and structure

- As programs age, their structure is degraded and they become harder to understand and change

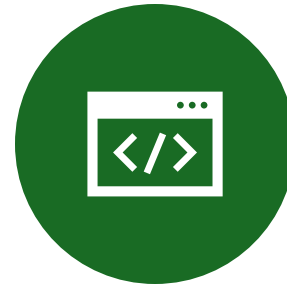
Real-world Factors Affecting Maintenance Cost

- The standard age of any software is considered up to **10 to 15 years**.
- **Older software**, which were meant to work on slow machines with less memory and storage capacity cannot keep themselves challenging against newly coming enhanced software on **modern hardware**.
- As technology advances, it becomes **costly to maintain** old software.
- Most maintenance engineers are newbie and use trial and error method to rectify problem.
- Often, changes made can easily hurt the original structure of the software, making it hard for any subsequent changes.
- **Changes are often left undocumented** which may cause more conflicts in future.

Software-end Factors Affecting Maintenance Cost



Structure of Software
Program



Programming Language



Dependence on
external environment

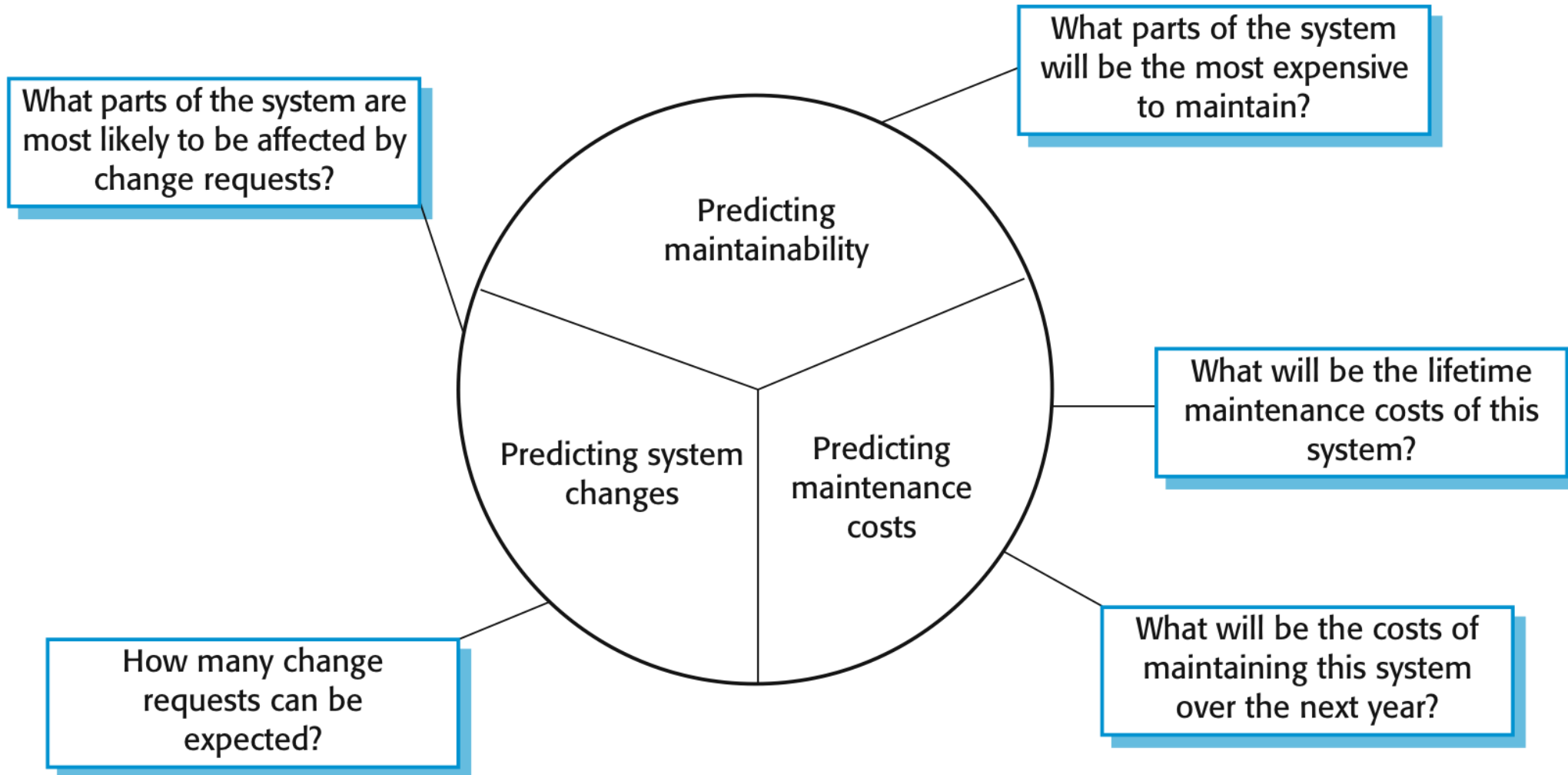


Staff reliability and
availability

Maintenance Prediction

- Maintenance prediction is concerned with assessing which parts of the system **may cause problems** and have high maintenance costs
 - Change acceptance depends on the maintainability of the components
 - Implementing changes degrades the system and reduces its maintainability
 - Maintenance costs depend on the number of changes

Maintenance Prediction



Seven Phases of IEEE maintenance process

Each of the seven activities have five associated attributes as follows:

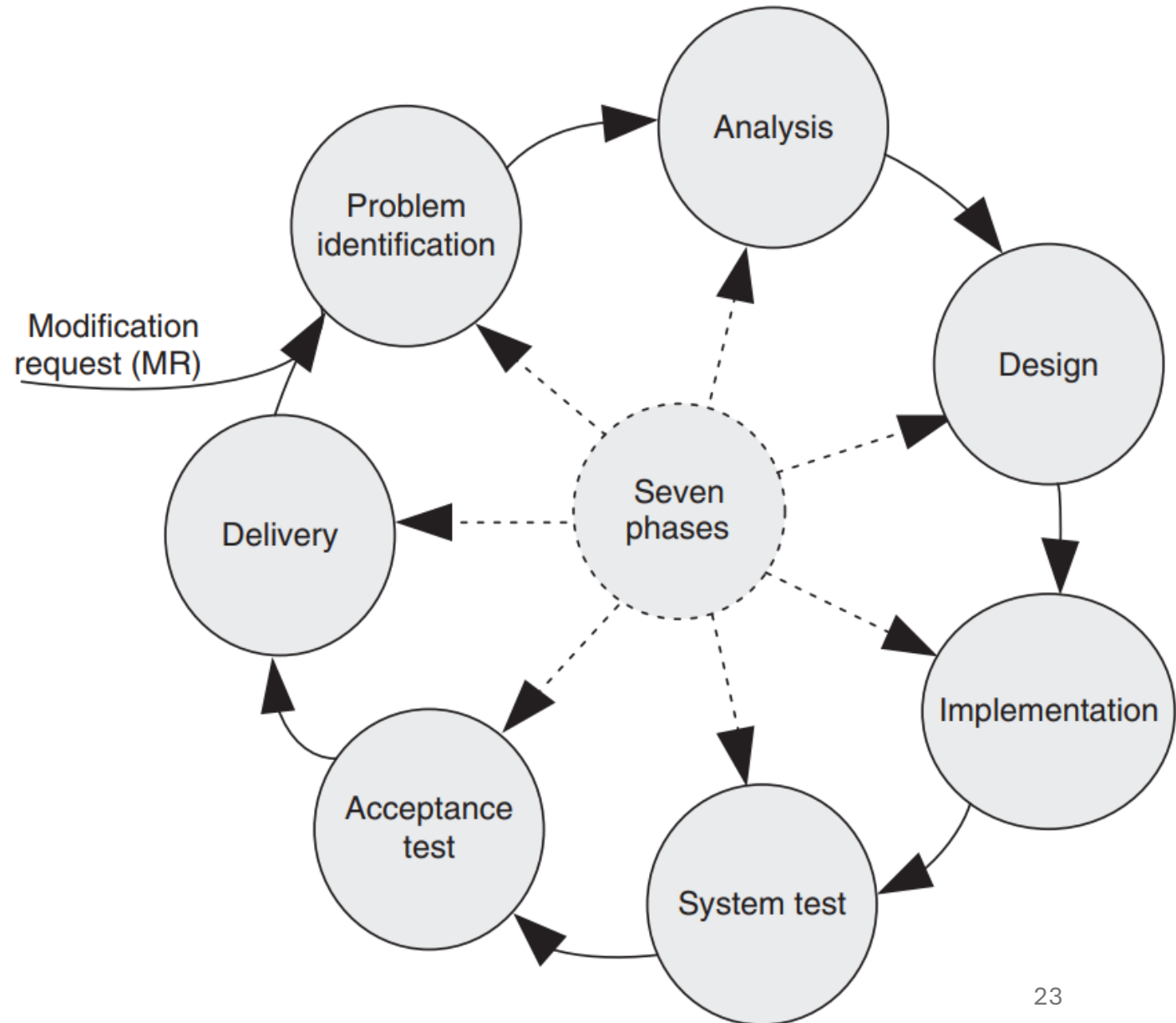
Activity definition. This refers to the implementation process of the activity.

Input. This refers to the items that are required as input to the activity.

Output. This refers to the items that are produced by the activity.

Control. This refers to those items that provide control over the activity.

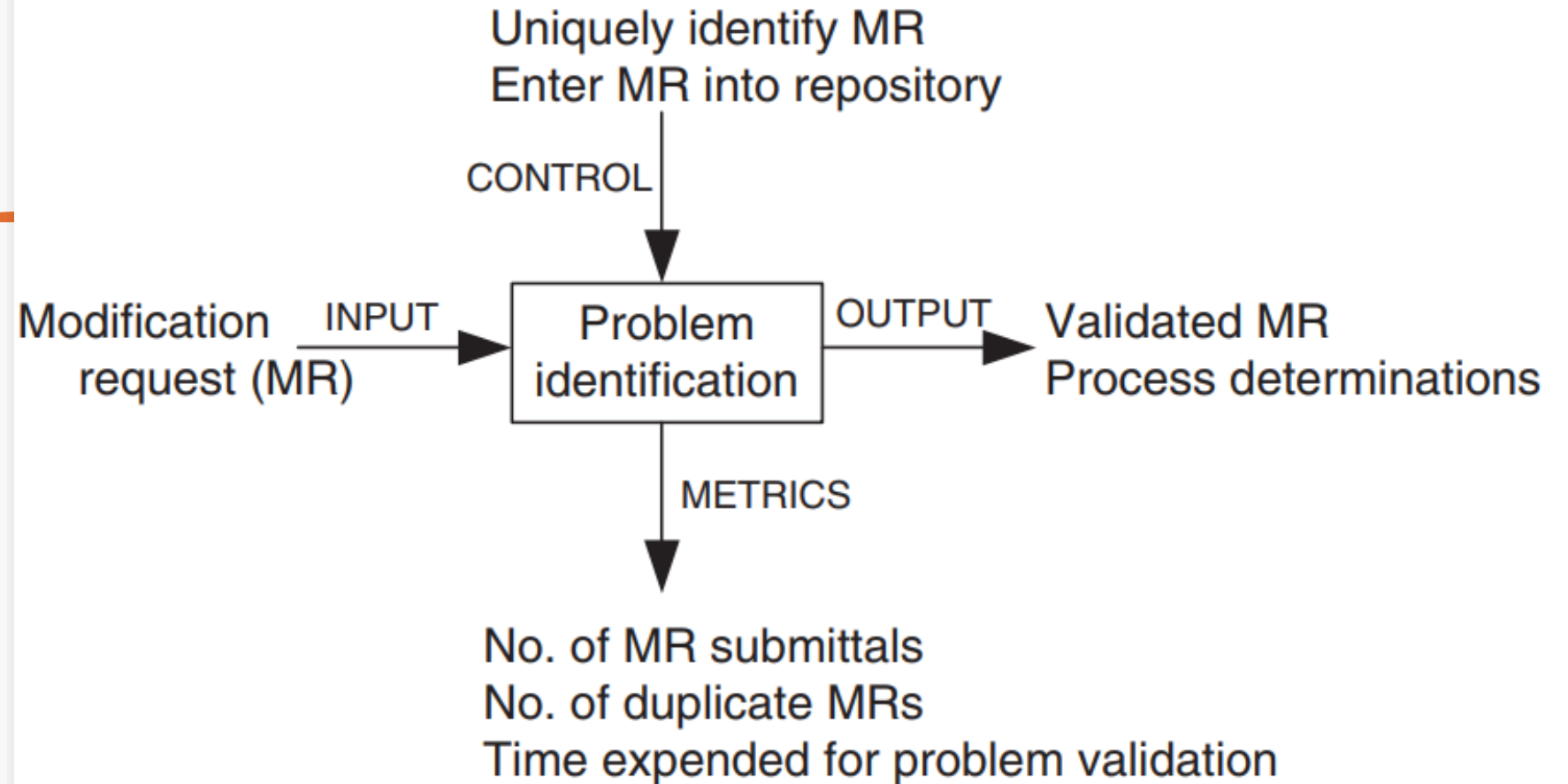
Metrics. This refers to the items that are measured during the execution of the activity



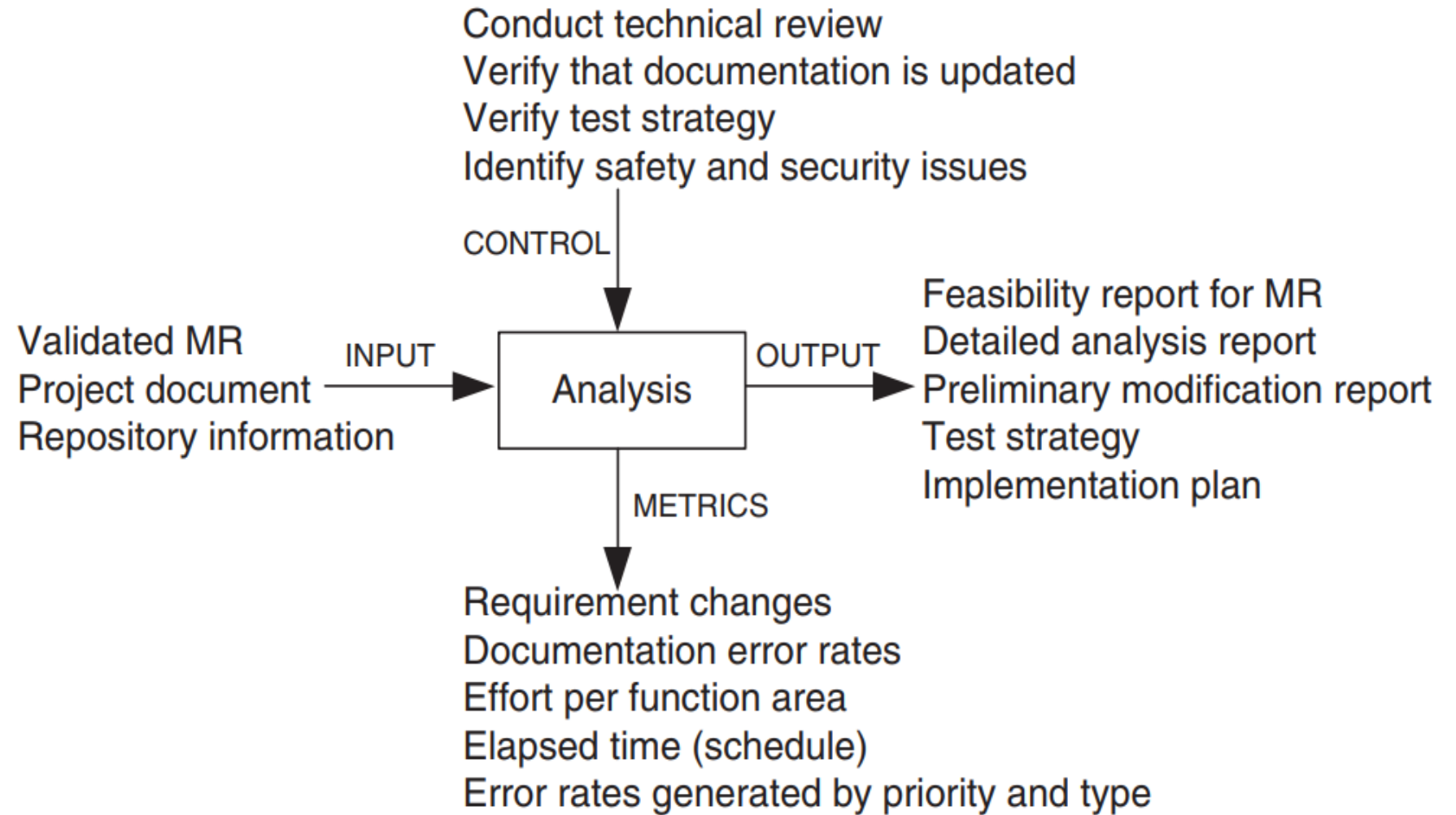
Cont..

- **Identification & Tracing** - It involves activities pertaining to identification of requirement of modification or maintenance. It is generated by user or system may itself report via logs or error messages. Here, the maintenance type is classified also.
- **Analysis** - The modification is analyzed for its impact on the system including safety and security implications. If probable impact is severe, alternative solution is looked for. A set of required modifications is then materialized into requirement specifications. The cost of modification/maintenance is analyzed and estimation is concluded.
- **Design** - New modules, which need to be replaced or modified, are designed against requirement specifications set in the previous stage. Test cases are created for validation and verification.
- **Implementation** - The new modules are coded with the help of structured design created in the design step. Every programmer is expected to do unit testing in parallel.
- **System Testing** - Integration testing is done among newly created modules. Integration testing is also carried out between new modules and the system. Finally, the system is tested as a whole, following regressive testing procedures.
- **Acceptance Testing** - After testing the system internally, it is tested for acceptance with the help of users. If at this state, user complaints some issues they are addressed or noted to address in next iteration.
- **Delivery** - After acceptance test, the system is deployed all over the organization either by small update package or fresh installation of the system. The final testing takes place at client end after the software is delivered.

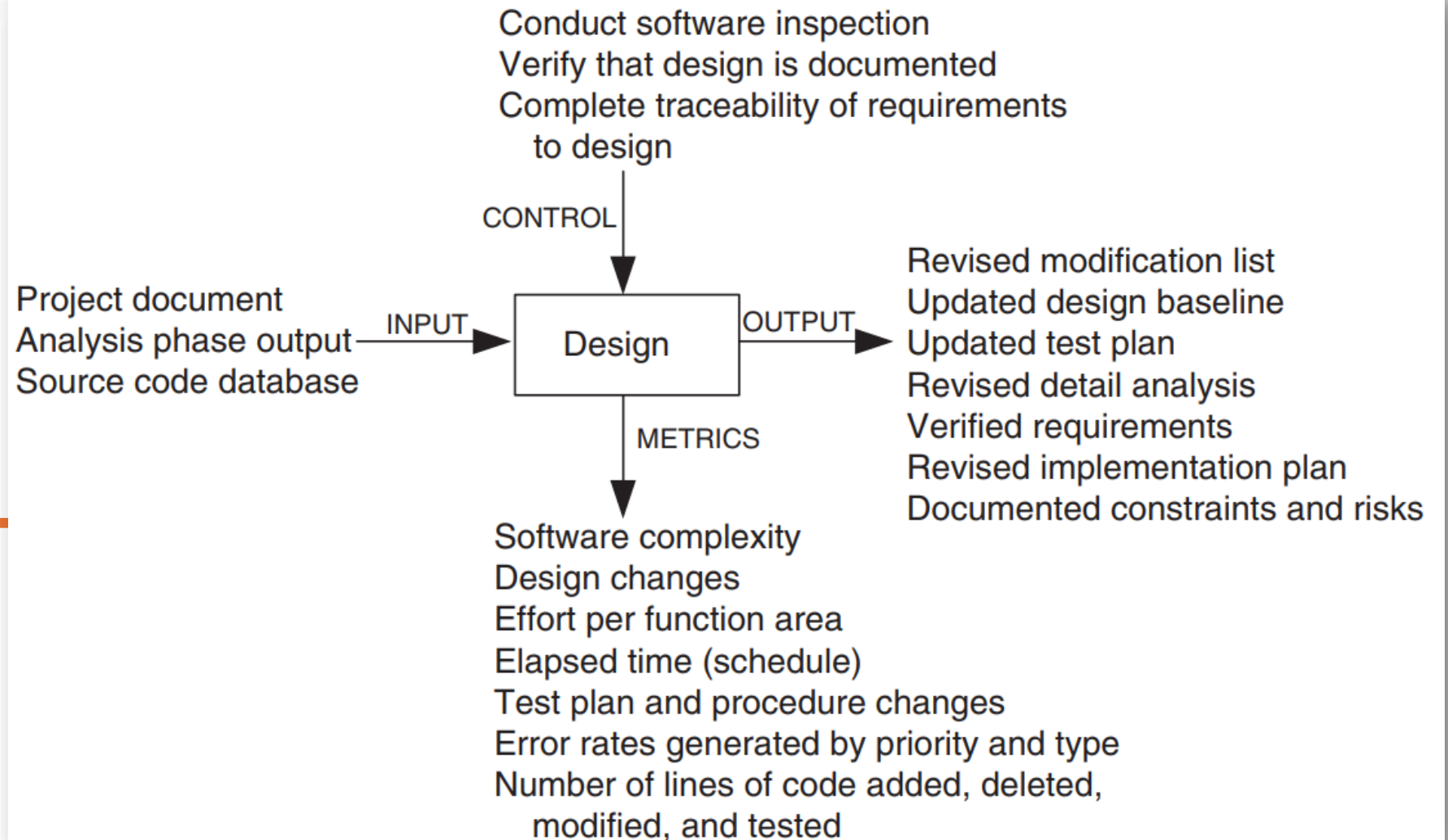
Problem identification phase



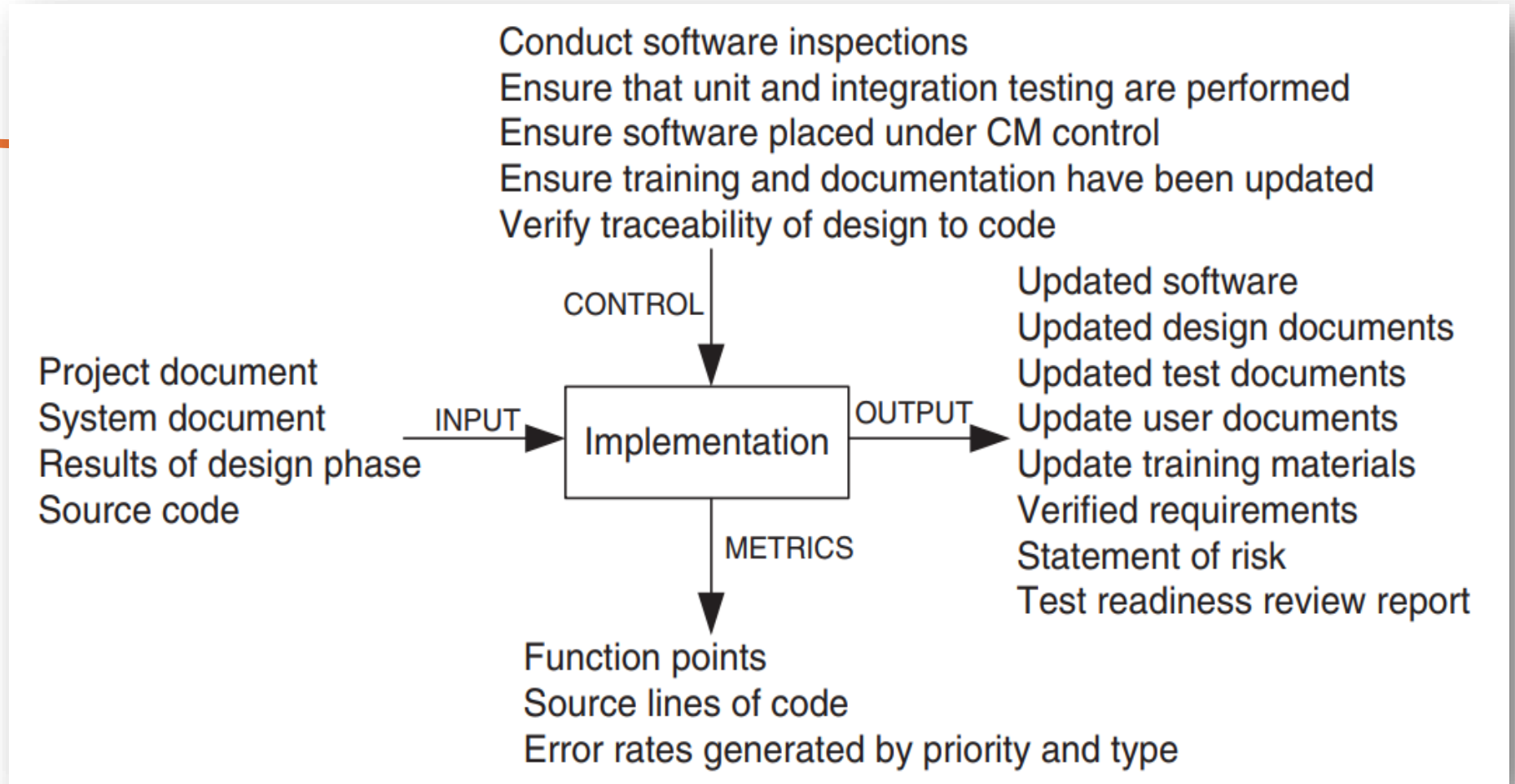
Analysis phase



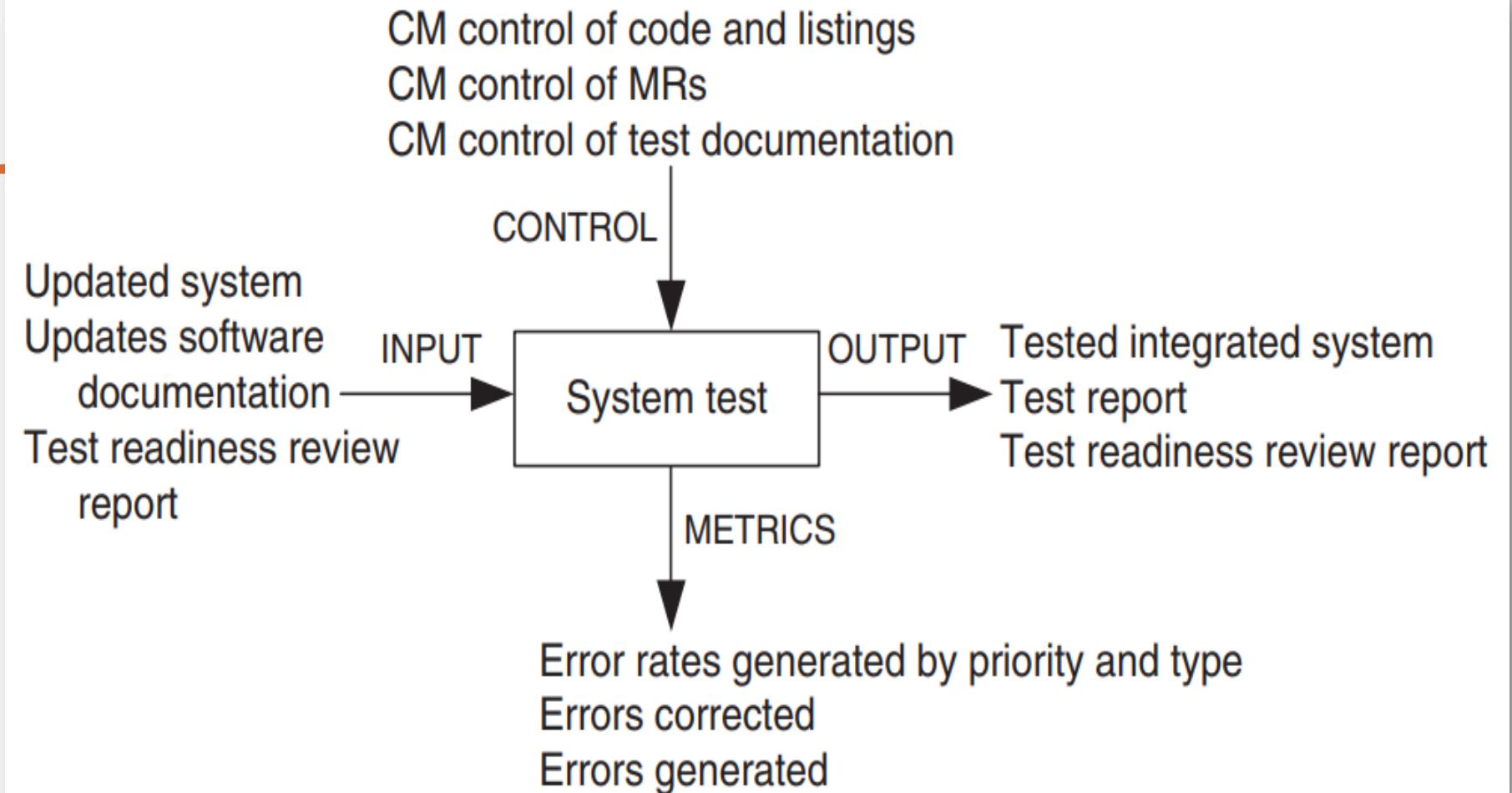
Design phase



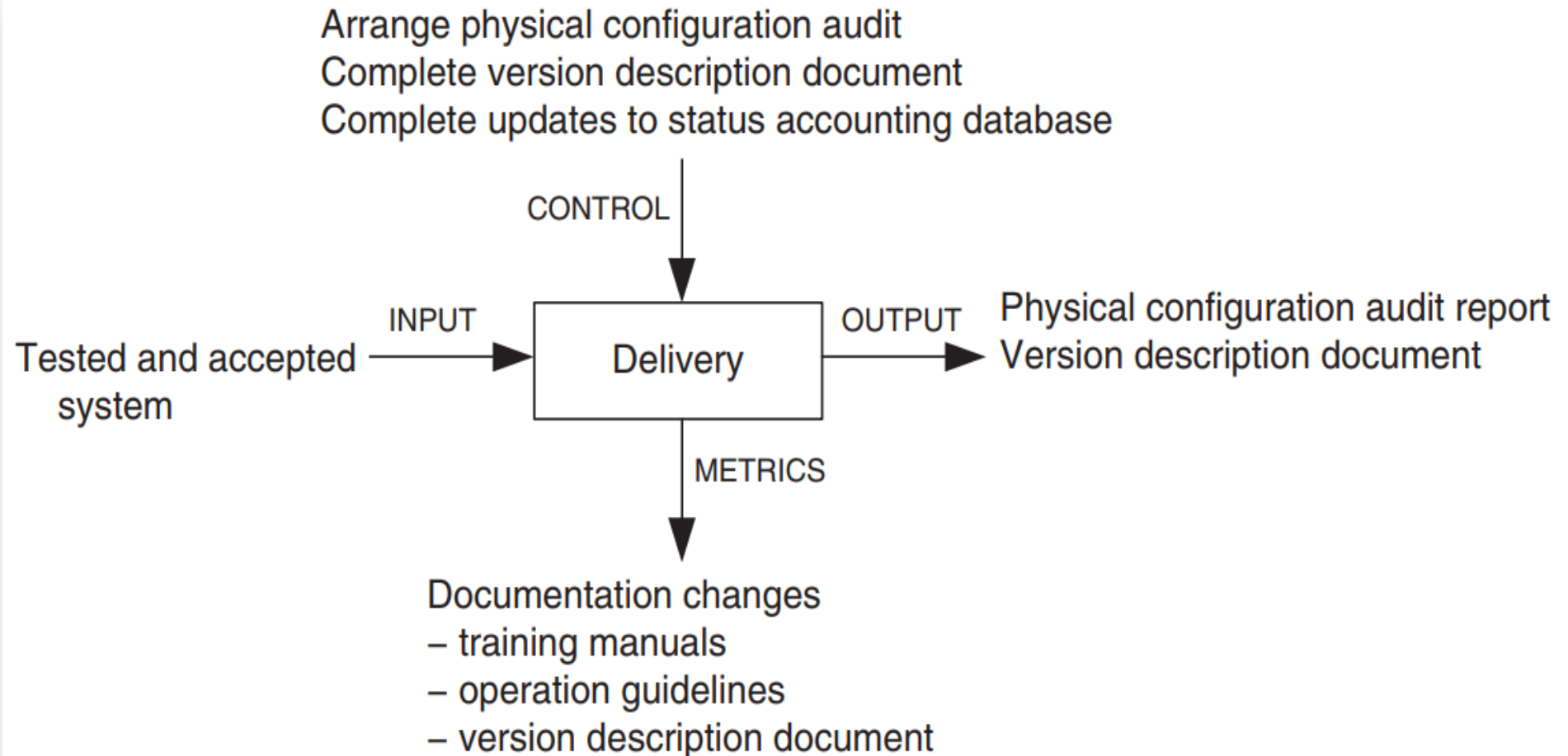
Implementation phase



System test phase

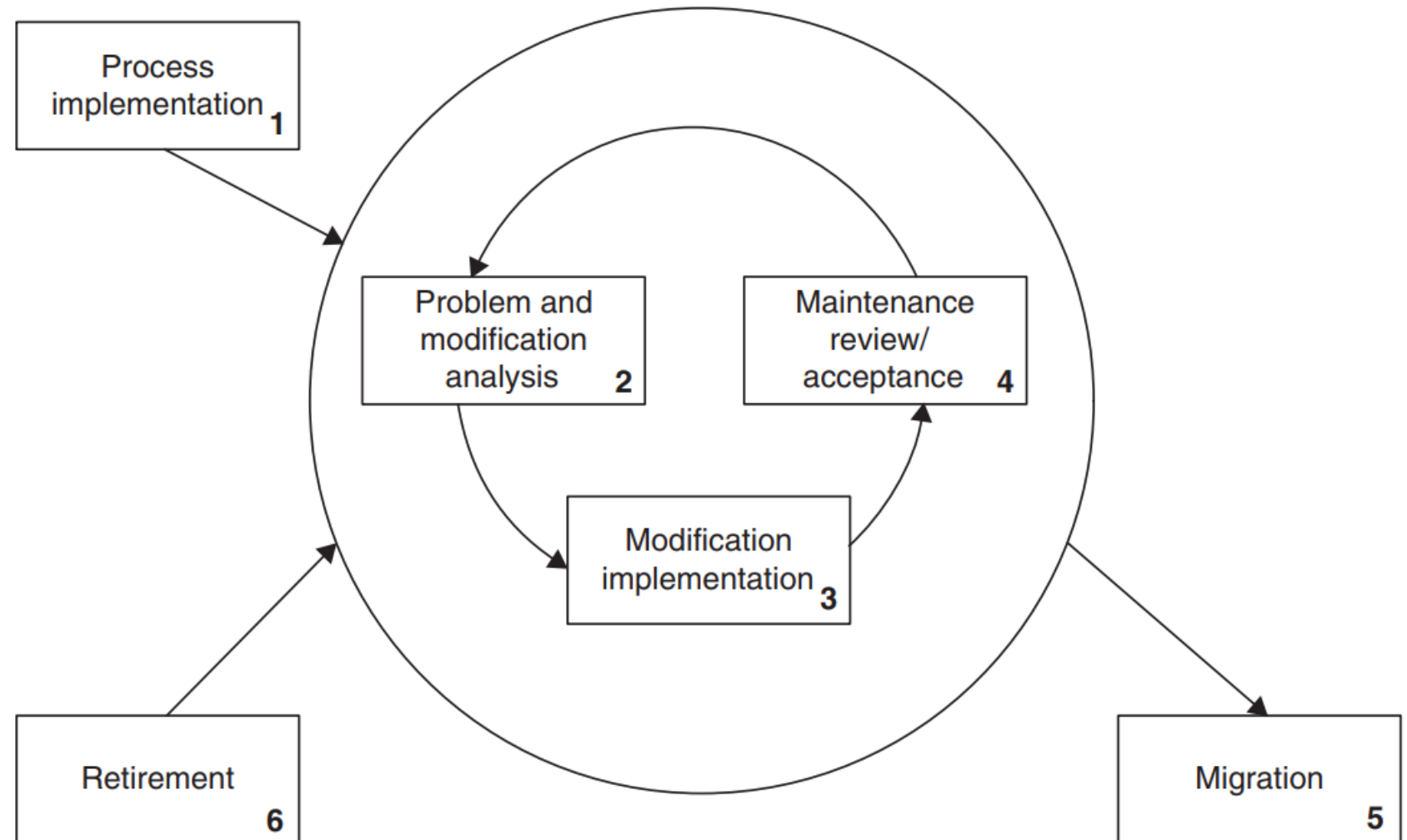


Delivery phase



ISO/IEC 14764 iterative maintenance process.

From © 2004 IEEE



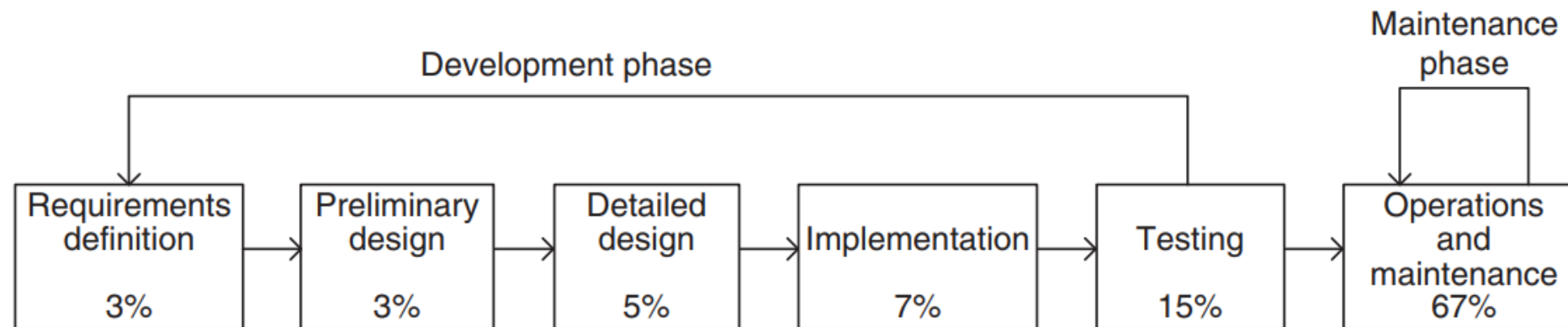
Evolution And Maintenance Models

(Selected)

Traditional SDLC model: Maintenance Cost

From © 1988 John Wiley & Son

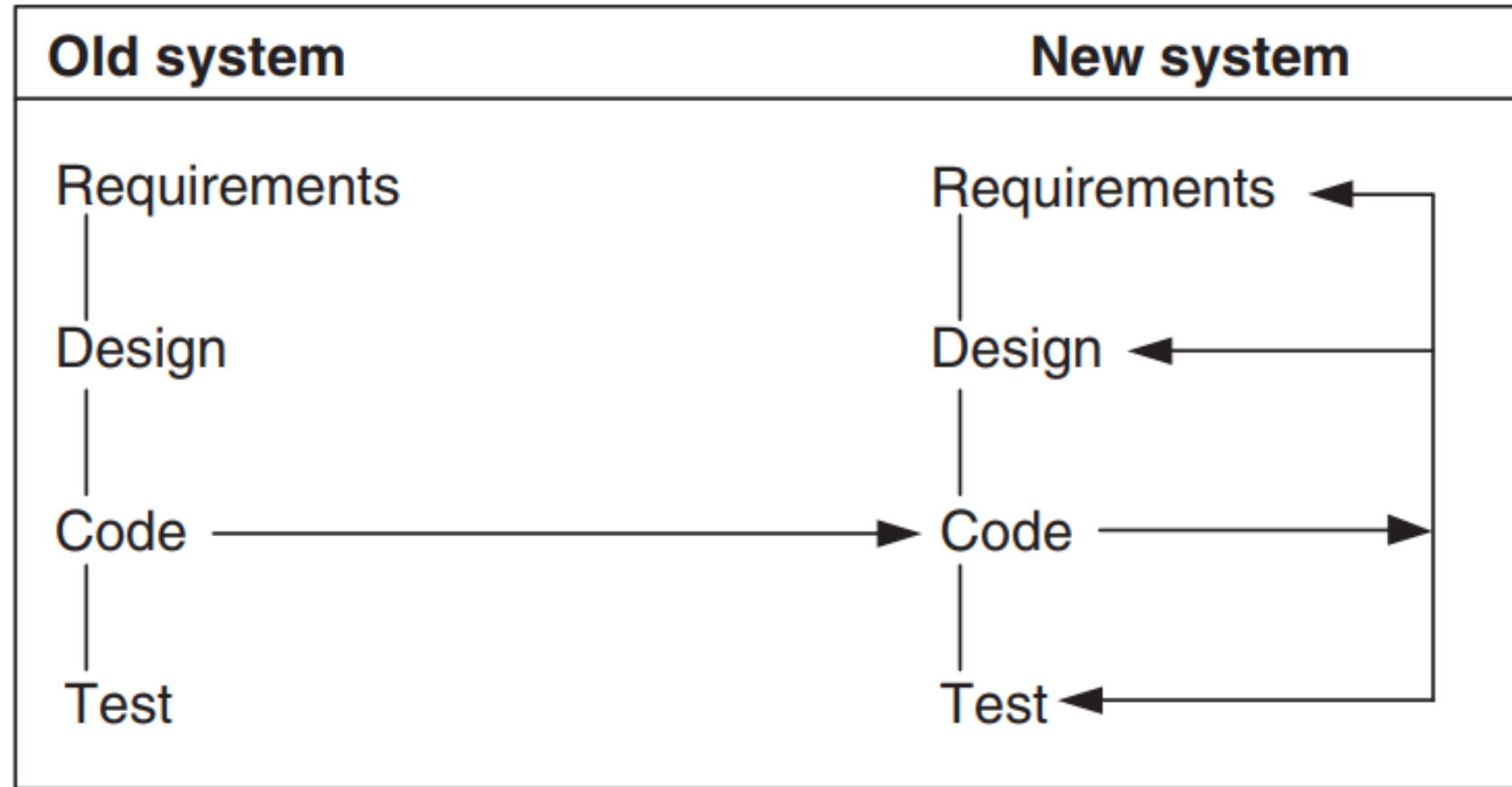
- **Constraints of an existing system.** Maintenance is performed on an operational system. Therefore, all modifications must be compatible with the constraints of the existing software architecture, design, and code.
- **Shorter time frame.** A maintenance activity may span from a **few hours to a few months**, whereas software development may span 1 or more years.
- **Available test data.** In software development, test cases are designed from scratch, whereas software maintenance can select a subset of these test cases and execute them as regression tests.



The quick fix model.

From Reference
© 1990 IEEE

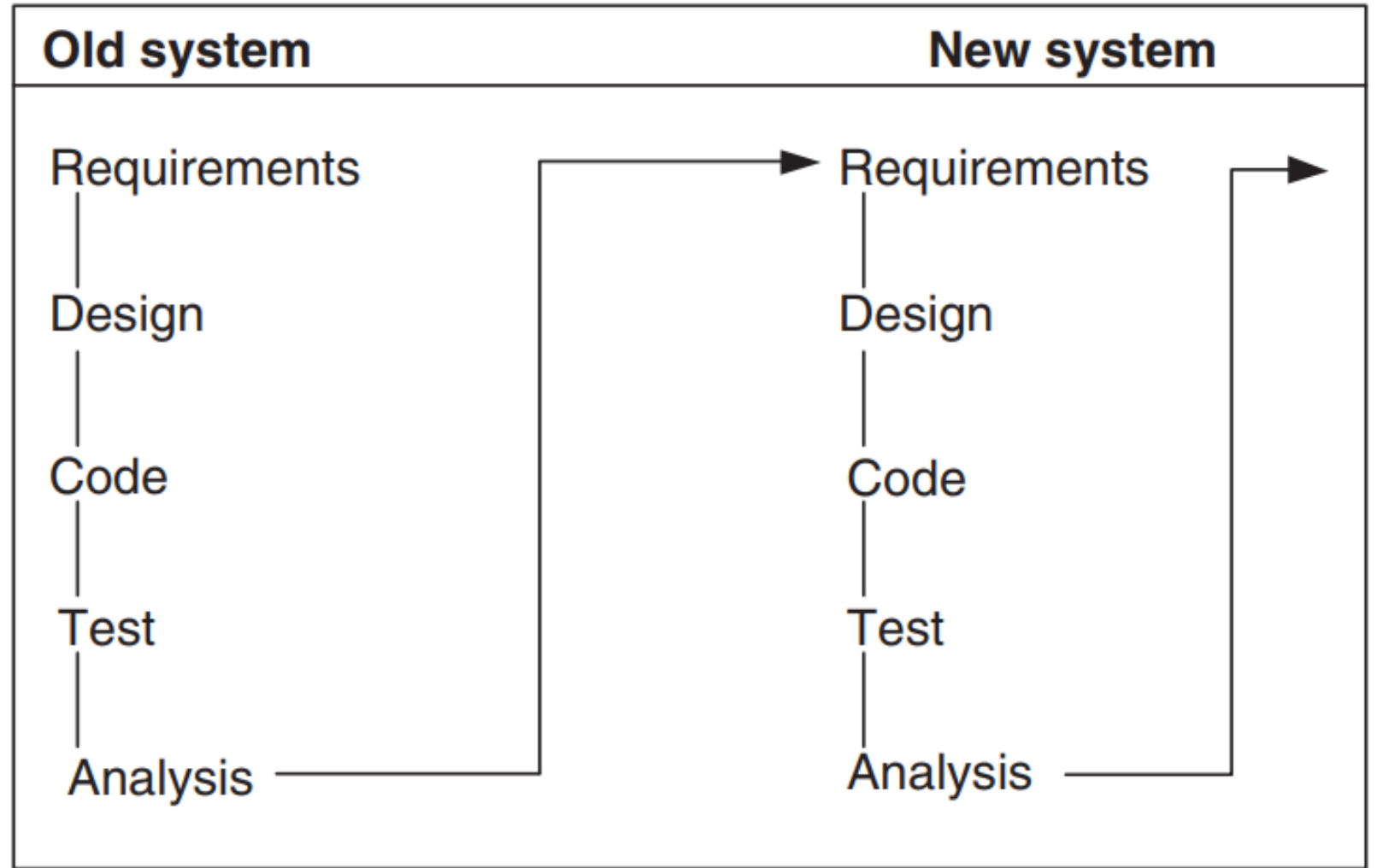
- Quick fix model. In this model, necessary changes are quickly made to the code and then to the accompanying documentation



The Iterative Enhancement Model.

From Reference © 1990 IEEE

- In this model, first changes are made to the highest level documents. Eventually, changes are propagated down to the code level

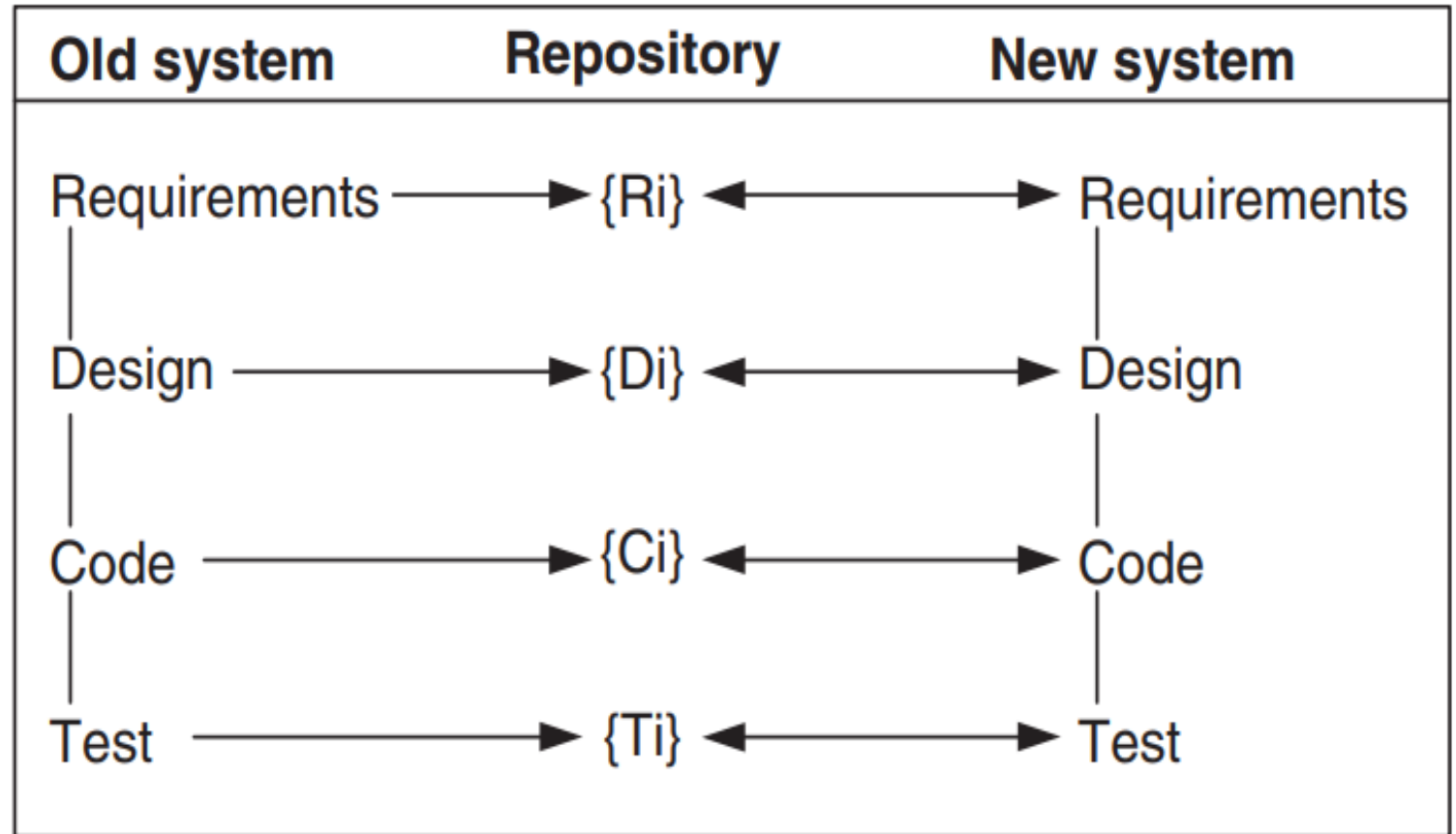


The Full Reuse Model

From Reference.

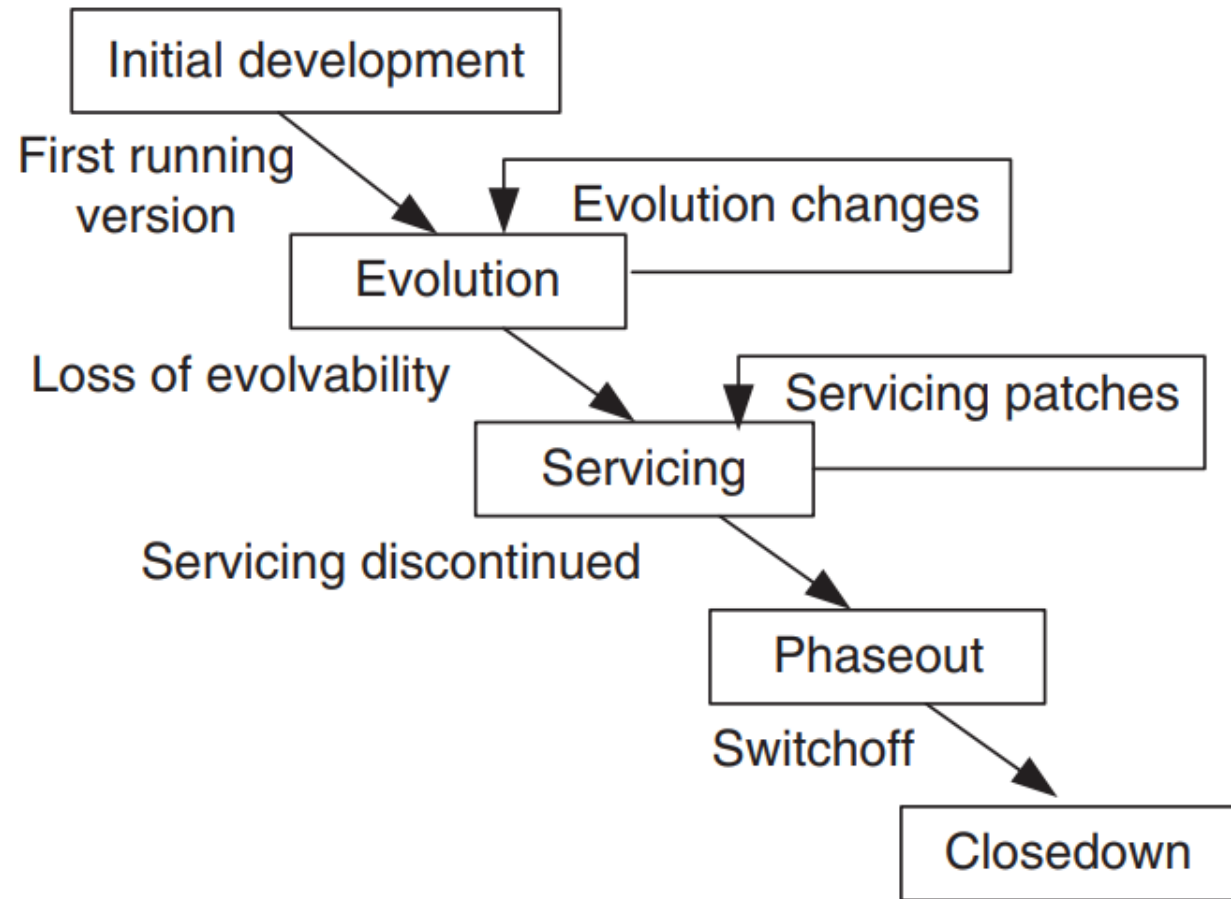
© 1990 IEEE

- Full reuse model. In this model, a **new system is built from components of the old system** and others available in the repository
- **Modify the old system components to support the new requirements**



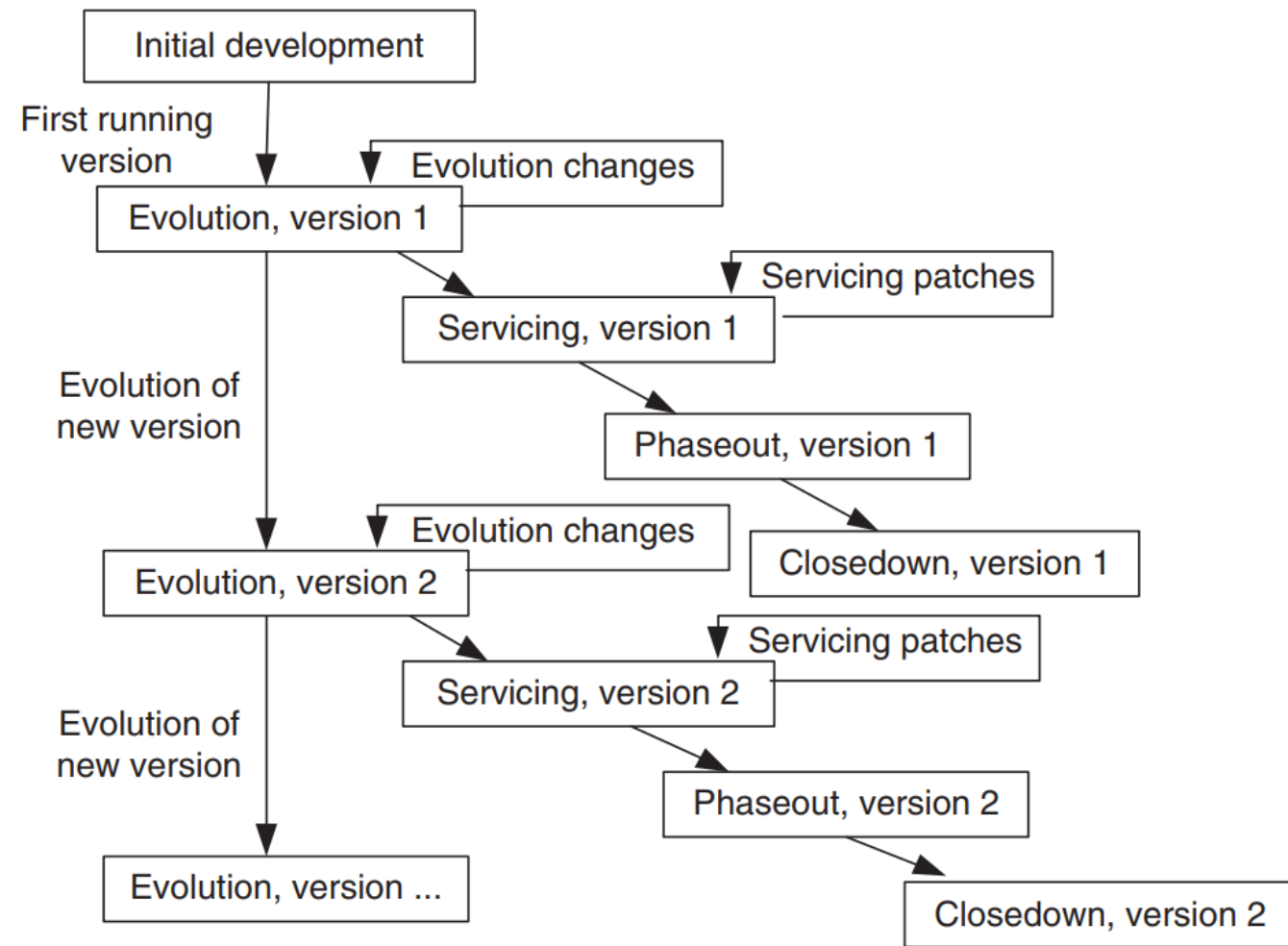
Staged Model For Closed Source Software (CSS)

- **Initial development.** Develop the first functioning version of the software.
- **Evolution.** The developers improve the functionalities and capabilities of the software to meet the needs and expectations of the customer.
- **Servicing.** The developers only fix minor and emergency defects, and no major functionality is included.
- **Phaseout.** In this phase, no more servicing is undertaken, while the vendors seek to generate revenue as long as possible.



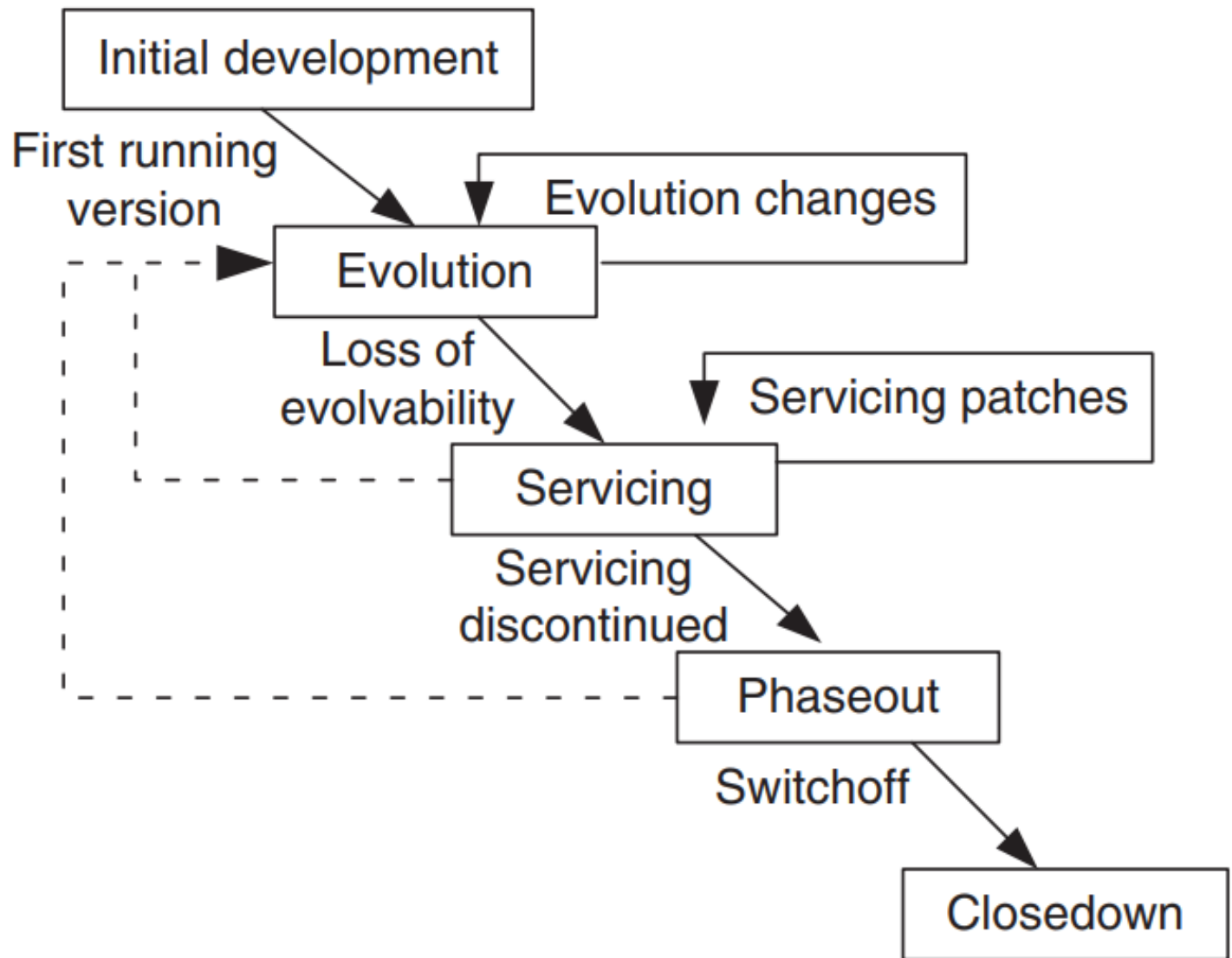
The versioned staged model for the CSS (Closed Source Code) life cycle.

From Reference
© 2000 IEEE



The staged model for the FLOSS (Open source) system.

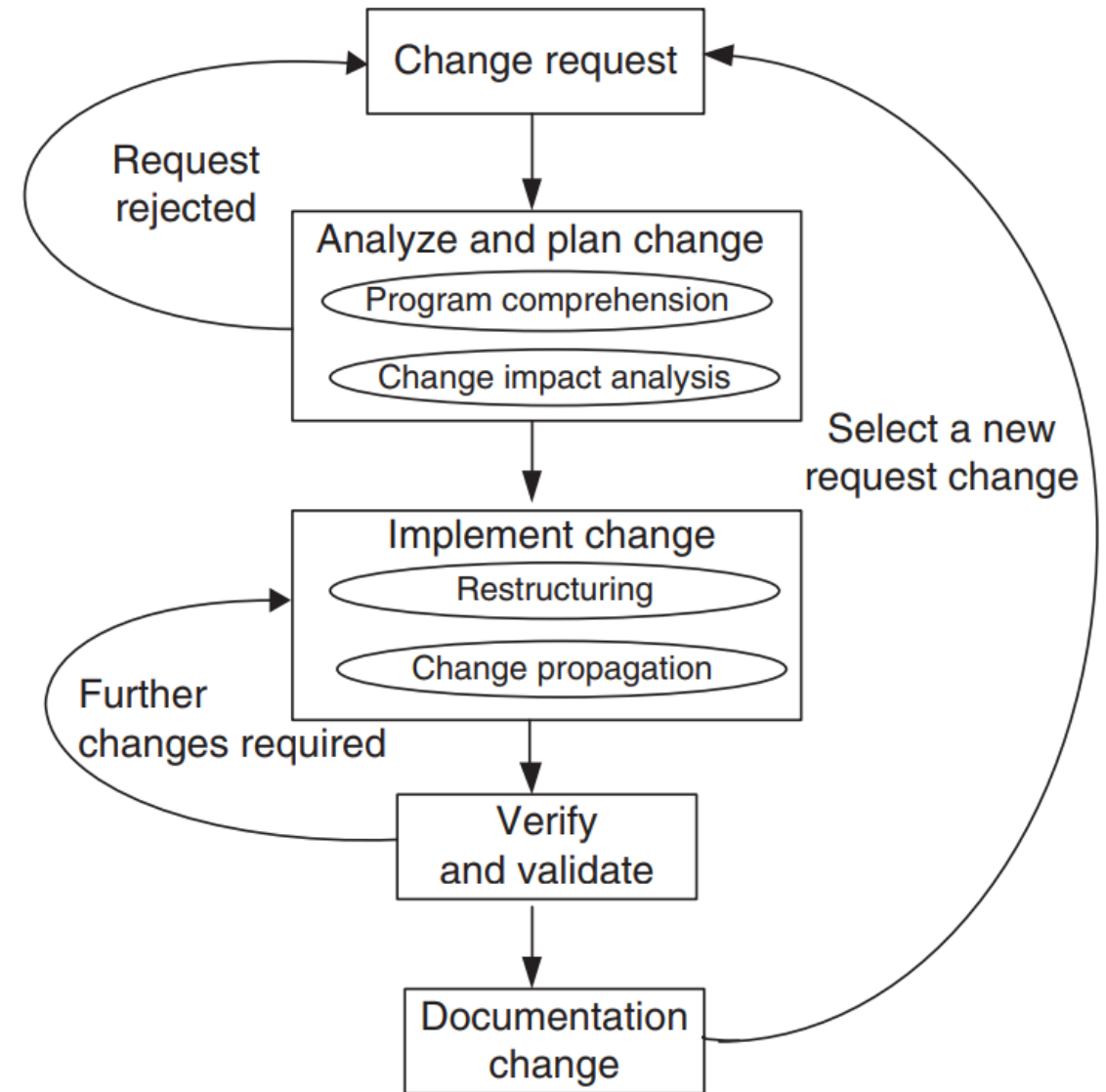
From Reference © 2007 ACM



Change Mini-cycle Model

From Reference
© 2008 Springer

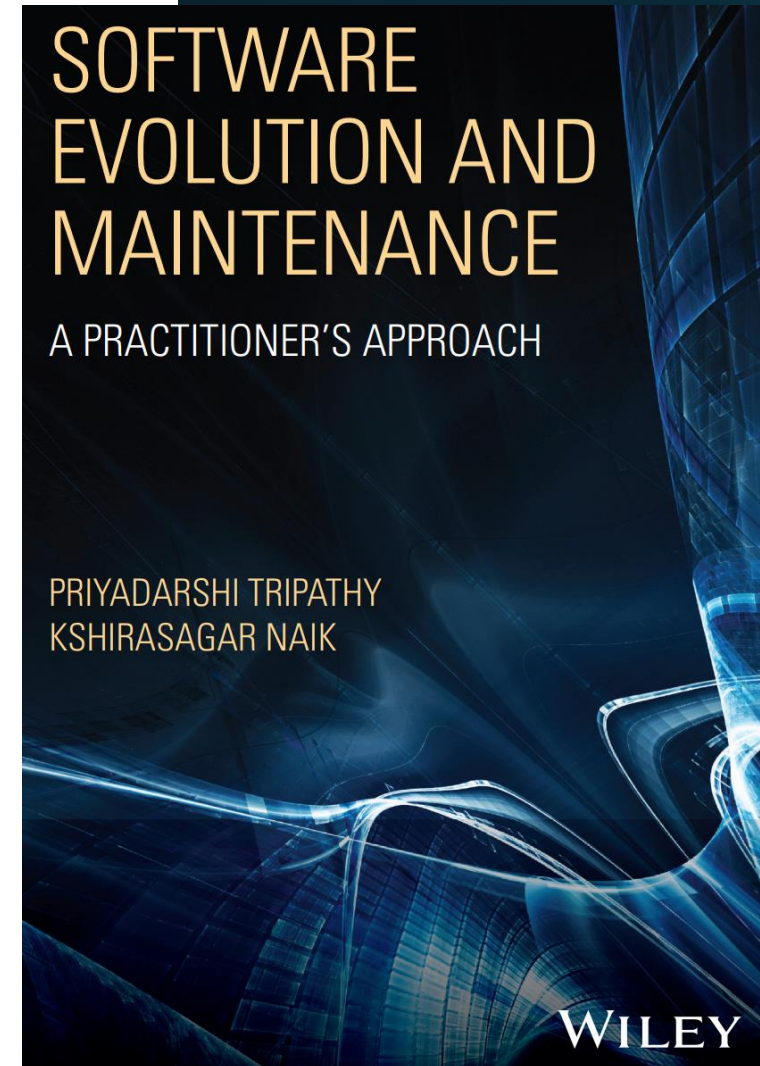
- A defect report describes the defect and software system actions that are out of line with requirements.
- An enhancement request describes a change to the requirements, functionality, or quality of the system.



Referece

Software Evolution and Maintenance, by Priyadarshi Tripathy & Kshirasagar Naik,

- Chapter 2: Taxonomy of Software Maintenance and Evolution
- Chapter 3: Evolution And Maintenance Models



Thanks for your attention!

Any Question?



Email me on : junaidakram@cuilahore.edu.pk